Experiment no.8

Aim : To understand the Static Analysis SAST process and learn to integrate Jenkins SAST to SonarQube.

Theory :

1. Introduction to Static Application Security Testing (SAST):

Static Application Security Testing (SAST) is a **white-box testing technique** used to identify vulnerabilities in the source code, bytecode, or binary files of an application **without executing** theprogram.

It helps developers detect security flaws early in the **Software Development Life Cycle (SDLC)** — typically during the coding phase — thus saving time, effort, and cost compared to fixing vulnerabilities after deployment.

---

2. Working Principle of SAST:

- The SAST tool **scans the source code** or compiled code of an application.
- It **analyzes data flows**, control flows, and dependencies to find issues such as:
  - SQL Injection
  - Cross-Site Scripting (XSS)
  - Buffer Overflows
  - Hard-coded passwords or secrets
  - Unvalidated inputs and insecure APIs
- The tool then provides **reports with line numbers, severity levels, and remediation suggestions**.

---

3. Advantages of SAST:

- Detects security flaws **early** in development.
- Does not require running the application (can test incomplete code).
- Integrates easily with CI/CD pipelines for **continuous security checks**.
- Improves **code quality** and adherence to **secure coding standards**.

---

4. SonarQube for Static Analysis:

SonarQube is an open-source platform used for **continuous inspection of code quality**. It performs **static code analysis** to detect:

- Bugs
- Code smells (poor design choices)
- Vulnerabilities and security hotspots

SonarQube supports multiple programming languages such as **Java, C++, Python, JavaScript, PHP**, and more. It provides a **dashboard** that visualizes code quality metrics, coverage, and risk areas, helping teams maintain clean and secure codebases.

---

5. Jenkins Integration with SonarQube (SAST Integration):

Jenkins, a popular **Continuous Integration (CI)** tool, can be used to **automate the SAST process** by integrating it with **SonarQube**.

Steps involved:

1. **Install and Configure SonarQube:**
    - Set up the SonarQube server.
    - Generate an authentication token for Jenkins.
2. **Install SonarQube Plugin in Jenkins:**
    - Go to *Manage Jenkins → Plugins → Available Plugins → SonarQube Scanner*.
3. **Configure SonarQube in Jenkins:**
    - Add SonarQube server details and token under *Manage Jenkins → Configure System*.
4. **Set Up Jenkins Pipeline/Job:**
    - Add a build step: "Execute SonarQube Scanner."
    - Define the sonar-project.properties file in your project repository.
5. **Run the Build:**
    - Jenkins will trigger SonarQube analysis during each build.
    - The analysis report is uploaded to the SonarQube dashboard automatically.

---

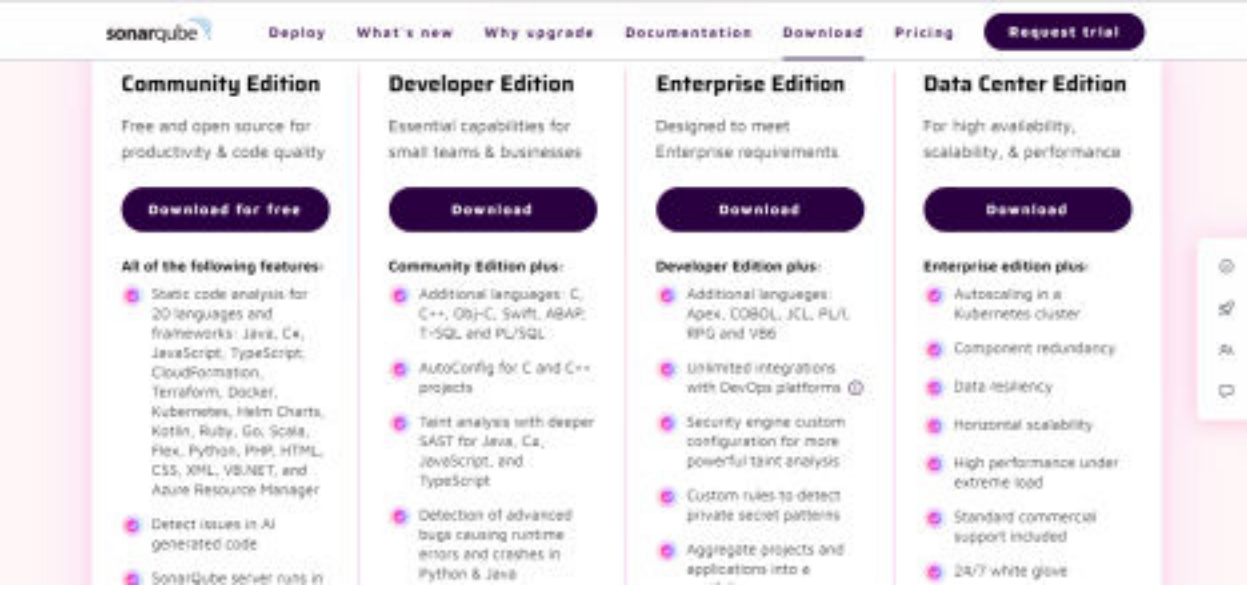
6. Benefits of Jenkins–SonarQube Integration:

- Enables **automated static analysis** in CI/CD pipelines.

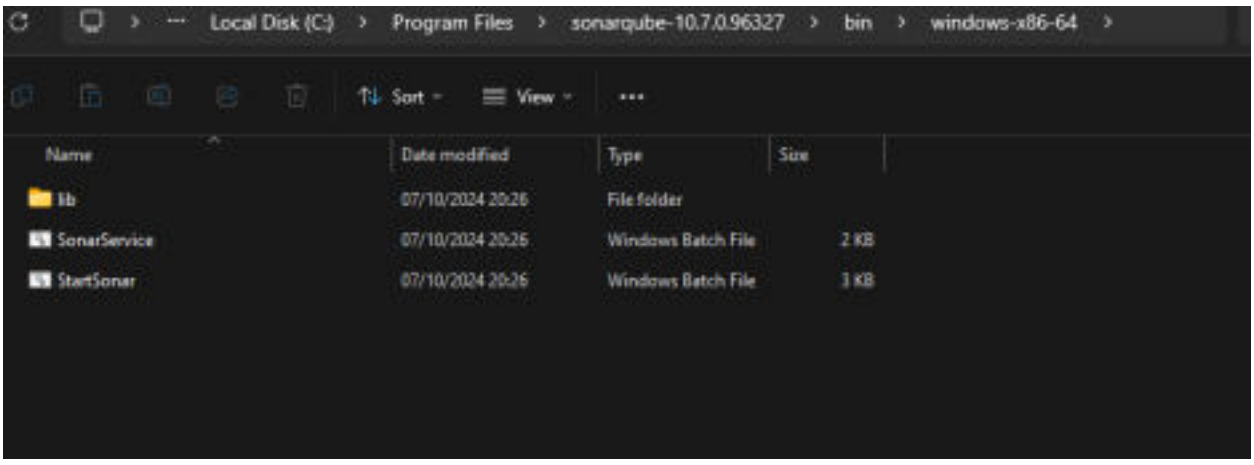
- Provides **real-time feedback** to developers about code security and quality.
- Helps enforce **secure coding practices** and compliance.
- Reduces manual effort and ensures **consistent quality checks** across builds.

Procedure:

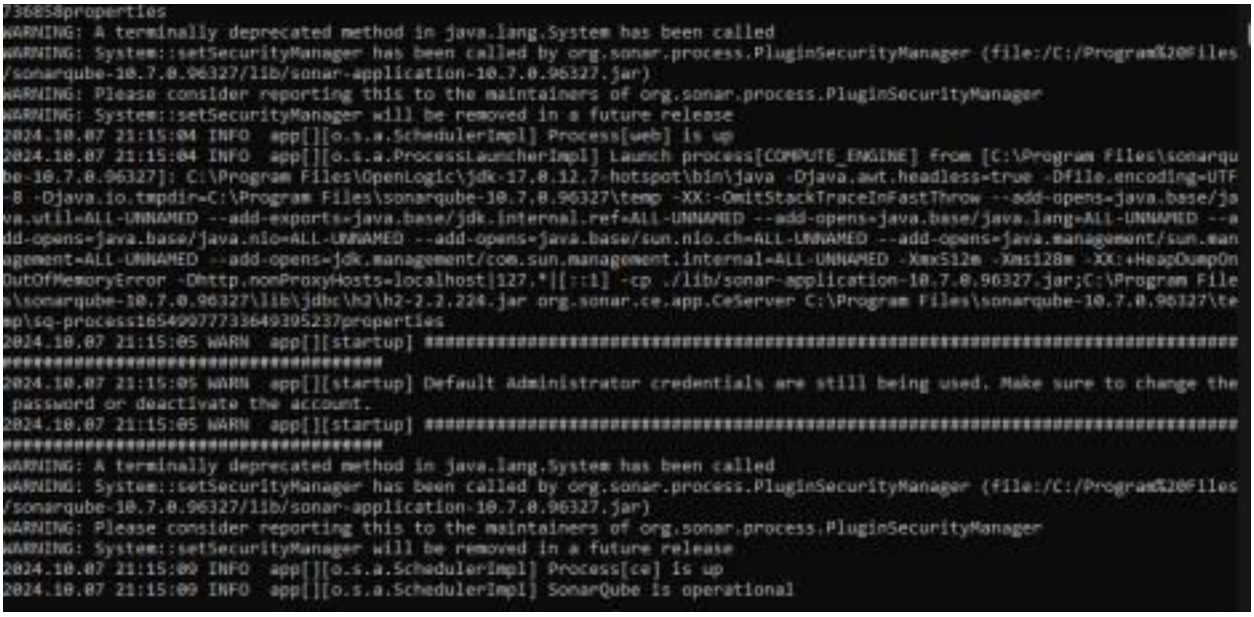
Step 1 :- Go To https://www.sonarsource.com/products/sonarqube/downloads and download Community Editions.



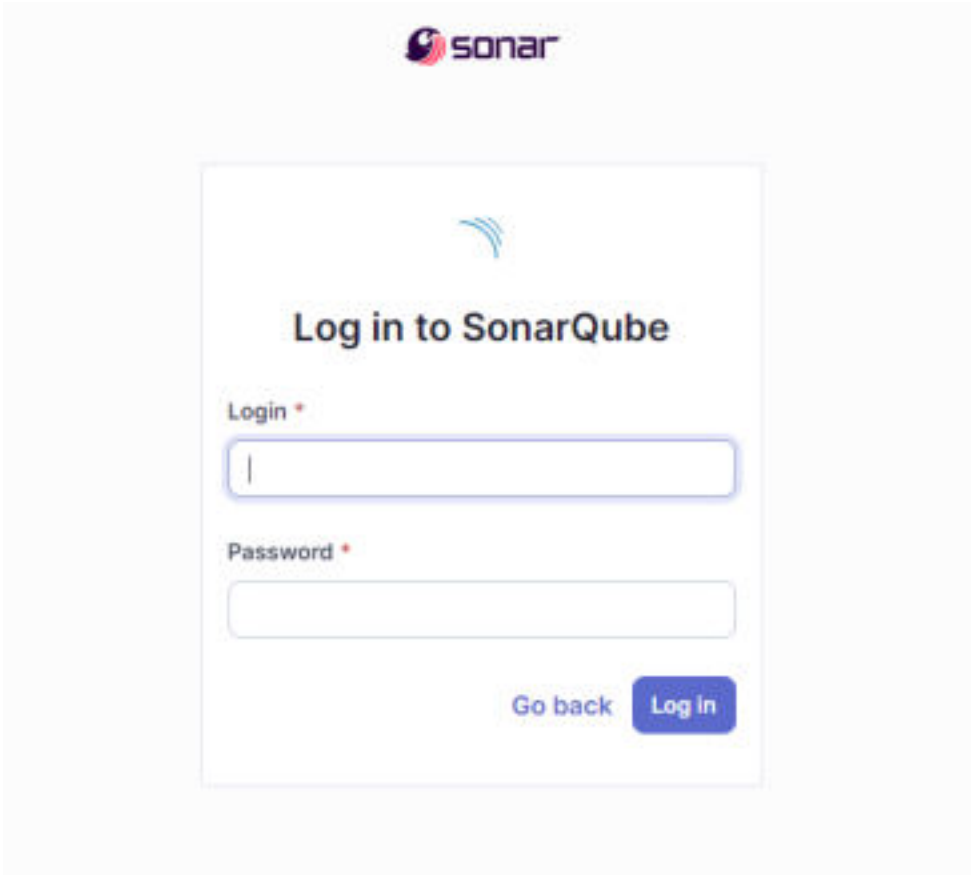
Step 2 :- Now Go To bin/windows-x86-64 and start StartSonar



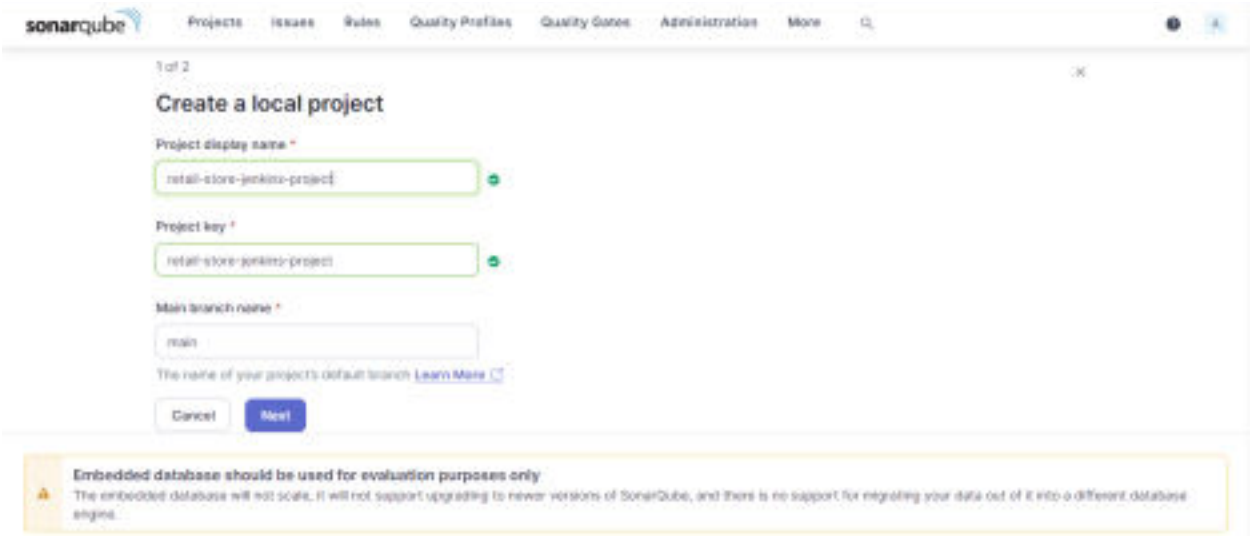
Step 3 :- You Will See Process Is Up & Sonarqube Is Operational



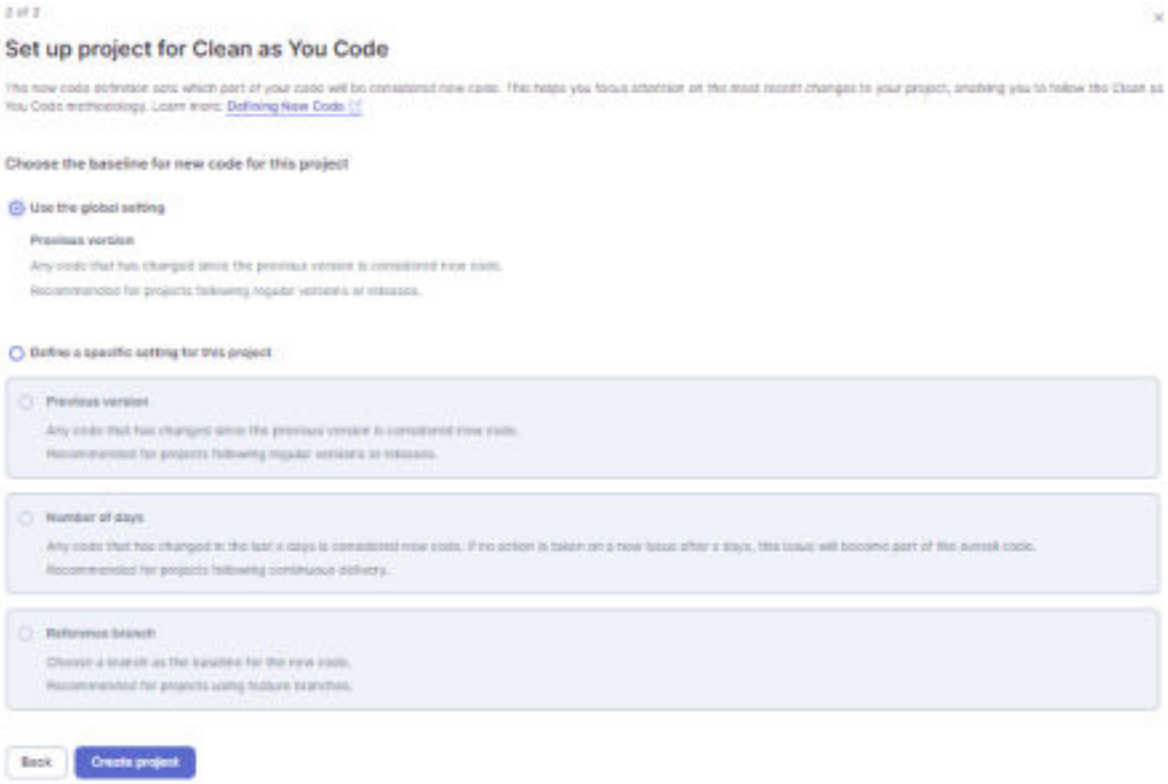
Step 4:- Go to http://localhost:9000 and login to Sonarqube



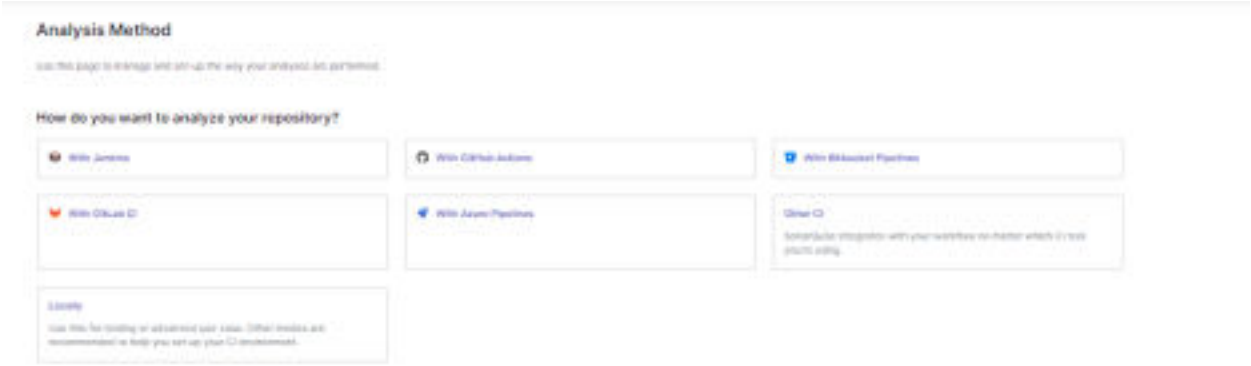
Step 5 :- Create A Local Project.



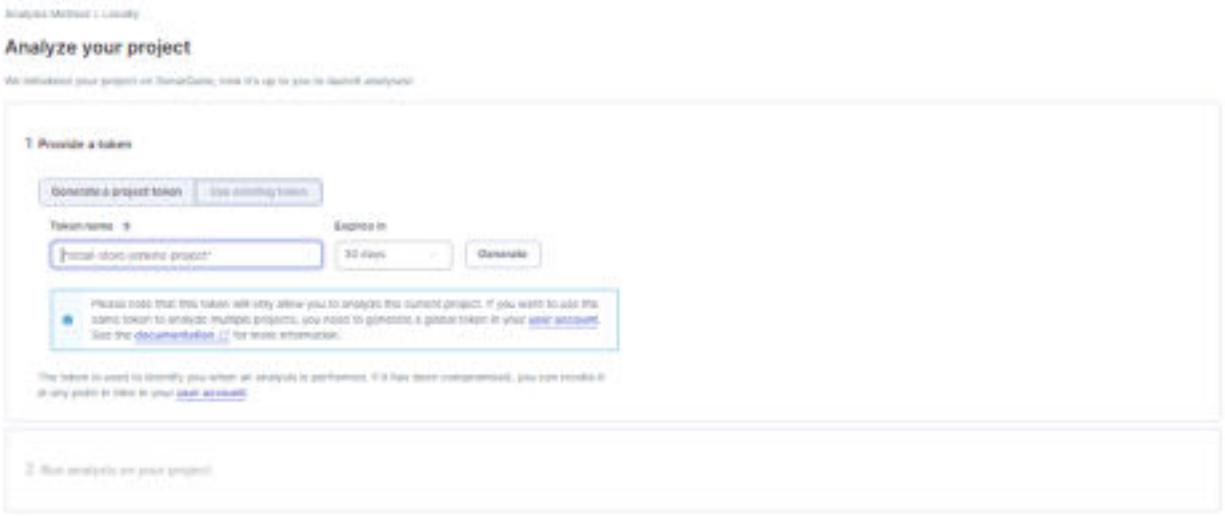
Step 6 :- Choose Global Setting As Baseline



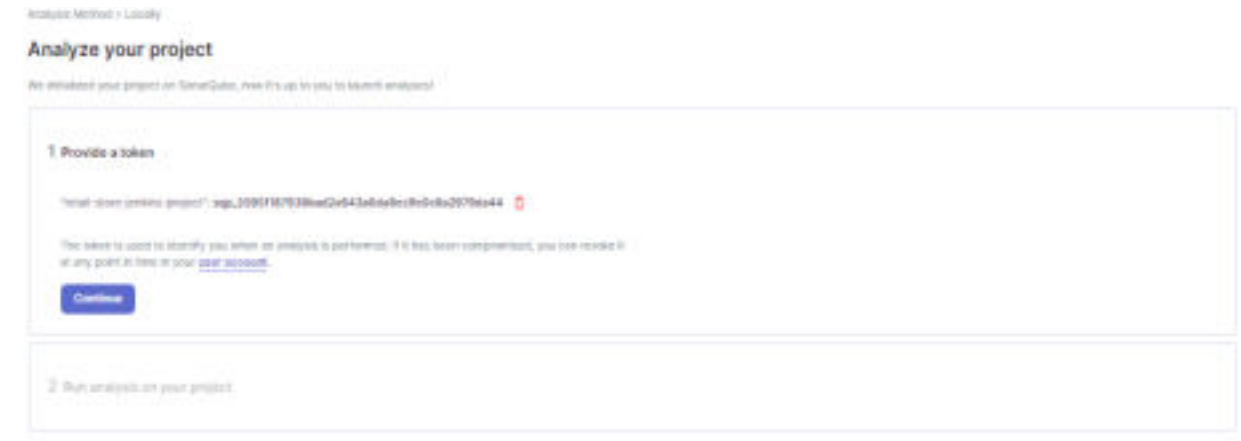
Step 7 :- Now Select Analysis Method As Locally



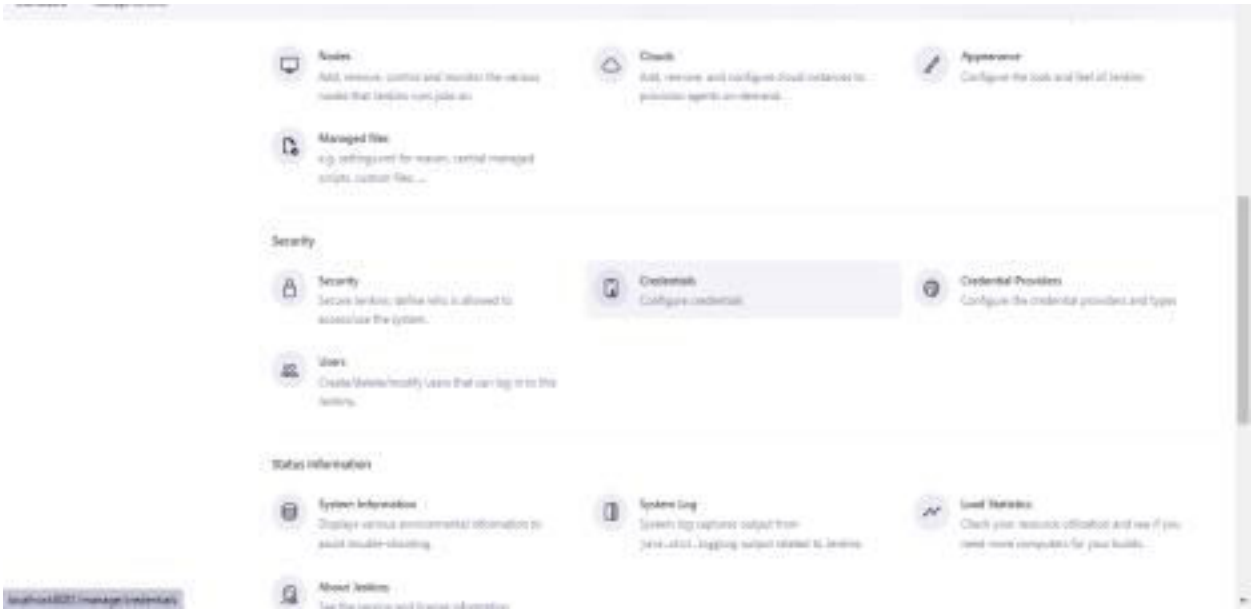
Step 8 :- Generate Token



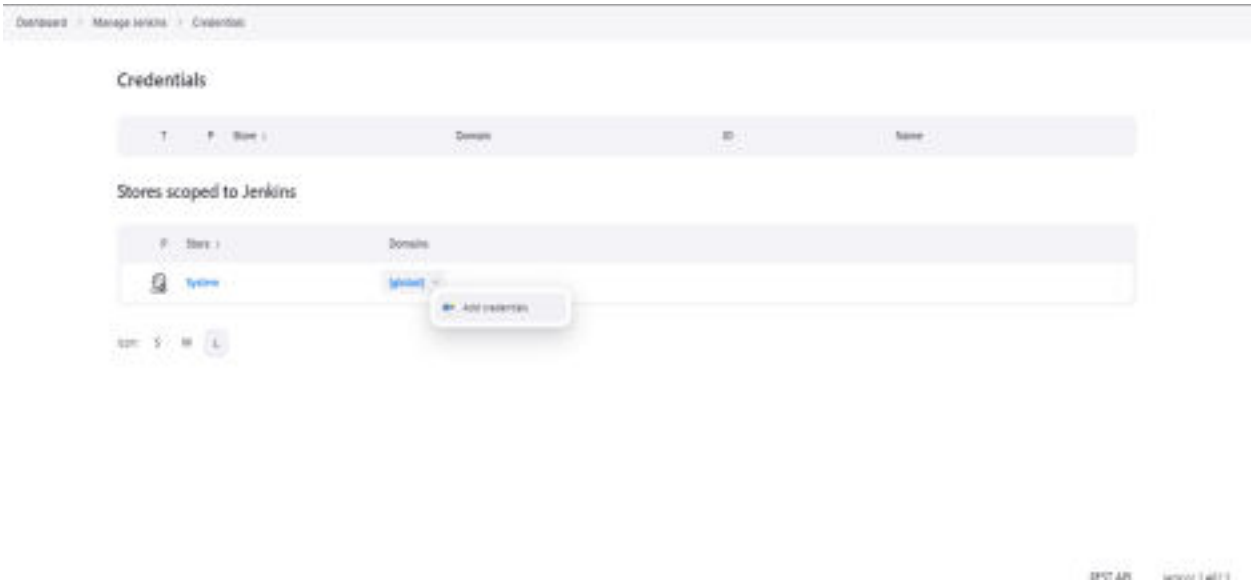
Step 9 :- After Generating Token Continue



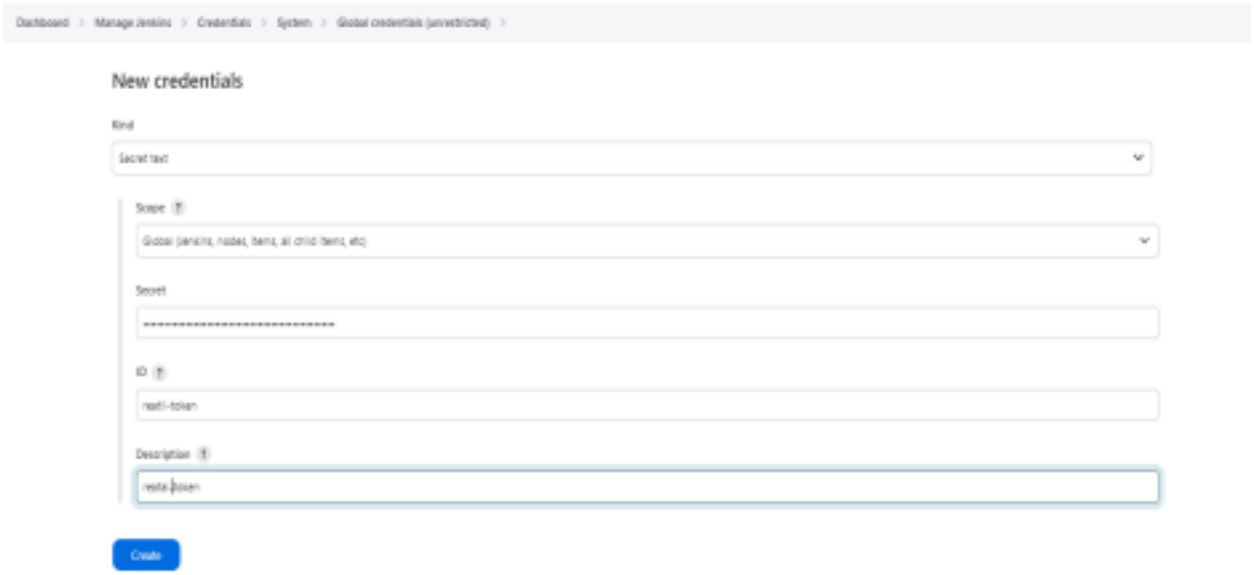
Step 10 :- Now In Jenkins Go To Manage Jenkins/Credential :-



Step 11 :- Now Go To Add Credential

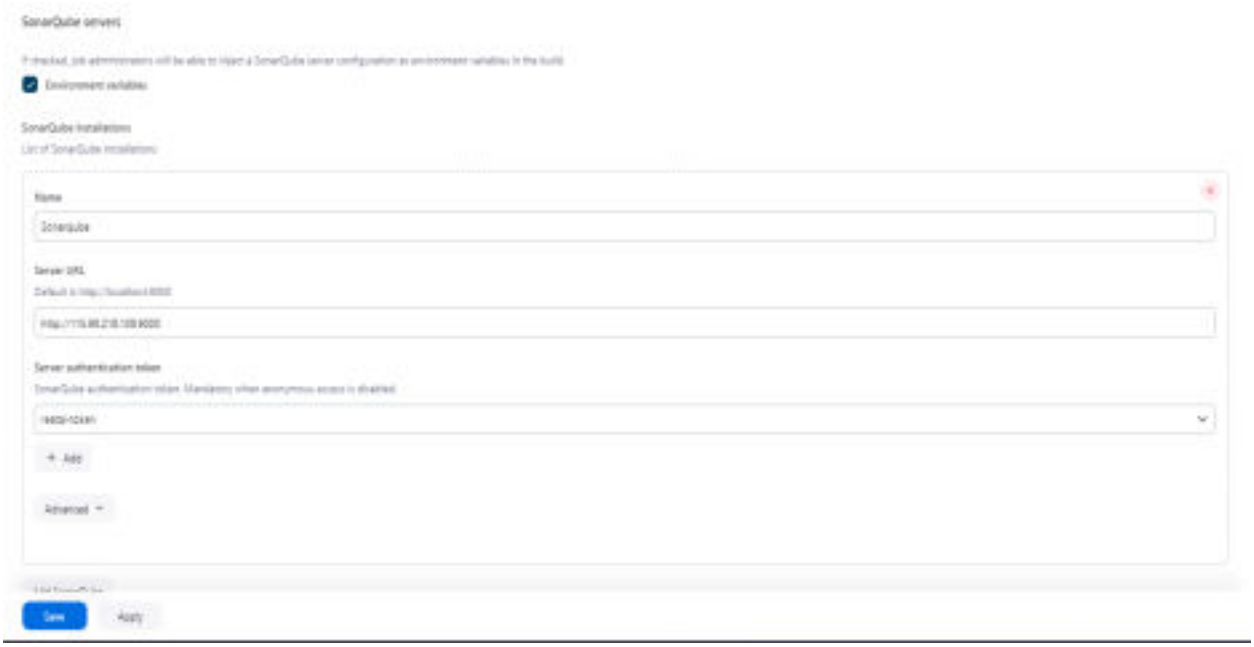


Step 12 :- Now Select “Kind” as Secret Text and Enter Your Token

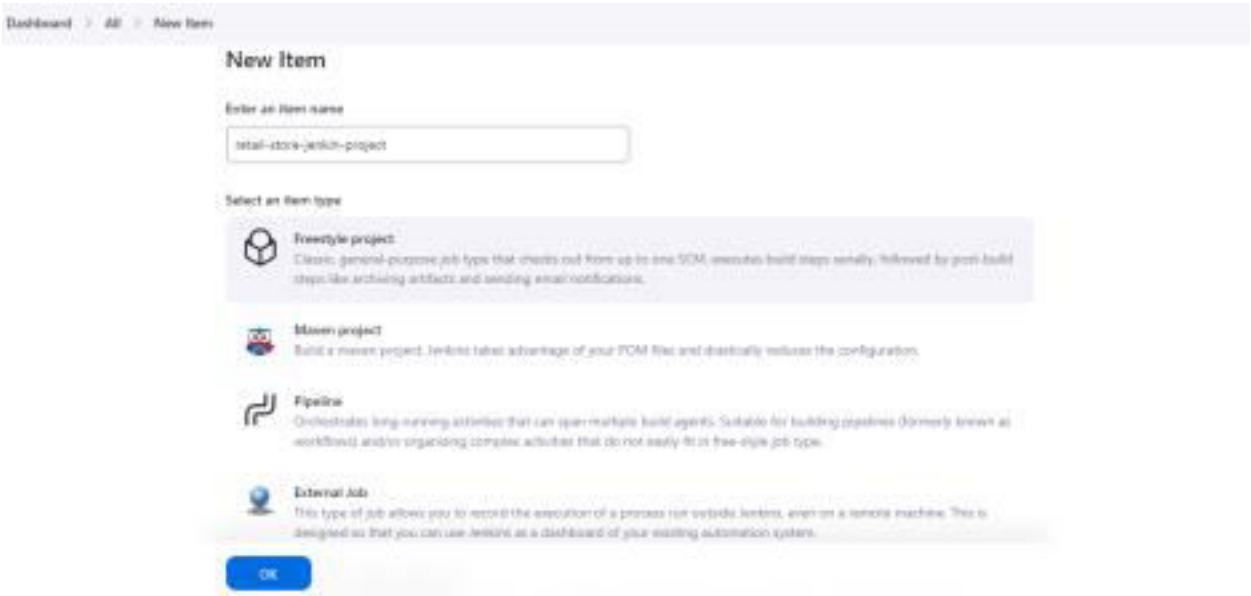


Step 13 :- Now You can see that Your Credential Is Created

Step 14 :- Go To Manage Jenkins/System/Sonarqube Server



Step 15 :- Create A New Project



Step 16 :- Add Your Git Repository

Step 17 :- In Build Environment Selecet Prepare Sonarqube Scanner Environment and enter your Token

Step 18 :- In Execute Sonar Scanner Add Your Analysis Properties

Step 19 :- Click On Build Now

Step 20 :- Console Output

Conclusion :

Integrating **Jenkins with SonarQube** allows organizations to automate the **SAST process**, ensuring that every code commit is checked for vulnerabilities and quality issues. This integration forms an essential part of **DevSecOps**, embedding security directly into the software development and delivery pipeline.